

IntCell *m; **ฉี 2 ขั้นตอน**

1. ระบุปฏิกิริยาการคอมพิวเตอรื จะทำการสร้างตัวแปร pointer ชื่อ (m) ขึ้นมา
จะขึ้นตามขนาดความจำ ที่ใช้ไว้กับค่า address ในหน่วยความจำตามโครงสร้าง
คลาส IntCell
m?
public
...
private
2. ระบุปฏิกิริยาการจะไปทำงาน Constructor function ก็คือฟังก์ชันที่จะ
สร้างฟังก์ชันที่สร้างตามรูปแบบตัวแปร
ฟังก์ชัน IntCell
IntCell (int initialValue = 0)
{ storedValue = initialValue;



IntCell class

```
/** A class for simulating an integer memory cell. */
class IntCell {
public:
    IntCell() { storedValue = 0; }
    IntCell(int initialValue) { storedValue = initialValue; }

    int read() { return storedValue; }

    void write(int x) { storedValue = x; }

private:
    int storedValue;
};
```

ไม่มีการแปร
พารามิเตอร์
overloading

IntCell() ฟังก์ชัน เมธอด

{ storedValue = 0; }

IntCell(int initialValue) ตัว b
{ storedValue = initialValue; }

int read() { return storedValue; }

void write(int x) { storedValue = x; }

private:
int storedValue; ปิด

จะสร้าง object ที่ main

```
int main()
{
    IntCell a;
    IntCell b(9);
    IntCell c(4,5);
    return 0;
}
```

constructor function
ที่มีตัวแปรพารามิเตอร์
2 ตัว

4

ใช้เงื่อนไขกันแต่แตกต่างกัน Overloading

1. ตัวแปร a เก็บไว้ในหน่วยความจำ ตามโครงสร้าง IntCell
2. ฟังก์ชันที่เรียกใช้ตามรูปแบบตัวแปร

= การใส่ assignment statement (ประโยคคำสั่ง)
ทำงานตามเงื่อนไขของตัวแปร

IntCell class

```

/** A class for simulating an integer memory cell. */
class IntCell
{
public:
    // 17 ข้อสงสัย / ข้อสงสัย
    // 1*/ explicit IntCell( int initialValue = 0 )
    // 2*/ : storedValue( initialValue ) { }
    // 3*/ int read( ) const
    // 4*/ { return storedValue; }
    // 5*/ void write( int x )
    // 6*/ { storedValue = x; }
private:
    // 7*/ int storedValue;
};

```

5

IntCell class interface IntCell.h

```

class IntCell
{
public:
    explicit IntCell( int initialValue = 0 );
    int read( ) const;
    void write( int x );
private:
    int storedValue;
};

```

6

IntCell class implementation

```

IntCell::IntCell( int initialValue ) : storedValue( initialValue )
{
}

int IntCell::read( ) const
{
    return storedValue;
}

void IntCell::write( int x )
{
    storedValue = x;
}

```

7

Program that uses IntCell

```

#include "IntCell.h"
int main( )
{
    IntCell m;

    m.write( 5 );
    cout << "Cell contents: " << m.read( ) << endl;

    return 0;
}

```

Program that use pointer point to IntCell

```

int main( )
{
    // 1*/ IntCell *m;
    // 2*/ m = new IntCell( 0 );
    // 3*/ m->write( 5 );
    // 4*/ cout << "Cell contents: " << m->read( ) << endl;

    // 5*/ delete m;
    // 6*/ return 0;
}

```

-> m เป็นหน่วยความจำที่ ... เก็บอยู่
 new มีสิ่งของที่จะมาวาง

MemoryCell template class

```
template <class Object>
class MemoryCell
{
public:
    explicit MemoryCell( const Object & initialValue = Object() )
    : storedValue( initialValue ) { }
    const Object & read() const
    { return storedValue; }
    void write( const Object & x )
    { storedValue = x; }

private:
    Object storedValue;
};
```

template class
Object
storedValue ← มีค่าเริ่มต้นคือ Object()

findMax function template

```
template <class Comparable>
const Comparable & findMax( const vector<Comparable> & a )
{
    /* 1*/      int maxIndex = 0;
    /* 2*/      for( int i = 1; i < a.size(); i++ )
    /* 3*/          if( a[ maxIndex ] < a[ i ] )
    /* 4*/              maxIndex = i;
    /* 5*/      return a[ maxIndex ];
}
```

template

15

17

Program that use MemoryCell

```
int main()
{
    MemoryCell<int> m1;
    MemoryCell<string> m2( "hello" );

    m1.write( 37 );
    m2.write( m2.read() + " world" );
    cout << m1.read() << endl << m2.read() << endl;

    return 0;
}
```

Object

Using findMax function template

```
int main()
{
    vector<int> v1( 37 );
    vector<double> v2( 40 );
    vector<string> v3( 80 );
    vector<intCell> v4( 75 );

    // Additional code to fill in the vectors

    cout << findMax( v1 ) << endl; // OK: Comparable = int
    cout << findMax( v2 ) << endl; // OK: Comparable = double
    cout << findMax( v3 ) << endl; // OK: Comparable = string
    cout << findMax( v4 ) << endl; // Illegal: operator< undefined

    return 0;
};
```

findMax

List

THE LIST ADT

- Ordered sequence of data items called elements
- $A_1, A_2, A_3, \dots, A_N$ is a list of size N
- size of an empty list is 0
- A_{i+1} succeeds A_i
- A_{i-1} precedes A_i
- position of A_i is i
- first element is A_1 called "head"
- last element is A_N called "tail"

Operations ?

THE LIST ADT

Operations

- PrintList
- Find
- FindKth
- Insert
- Delete
- Next
- Previous
- MakeEmpty

Link list

List<int> a; **ตัวอ่าน**

List < int > b; ตัวพิมพ์

`b = a.zeros()`; **เปลี่ยน** **เคอร์เซอร์** **ไปหาจุด**

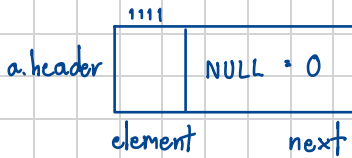
a. insert(3,b); put into list

b. advance (); เลื่อนเคอร์เซอร์ไปทางข้างหน้า

```
printList(a);
```

ไบนารี รวบรวมกัน แล้วให้มองไปง

NULL = 10 ในทศนิยมที่ 6



Find (ส่วที่ ๓) อยู่ตำแหน่งที่ ๑๑

findKth (တပ်မတော်) ကာကွယ်ရေးတပ်မတော်

ข้อ ๖๕๔ / ข้อจำกัด ของ List

1. เก็บ / คบข้อมลทำงานได้
2. คำนวณเพื่อที่จะหาผลรวม

การถ่ายภาพขาว

1. ทราบองค์ประกอบของวิสาหกิจชุมชน ลำดับขั้นที่
2. ไม่จำเป็นต้องเกี่ยวข้องกับสหกรณ์
3. ไม่สับสนกับวิสาหกิจชุมชน

special cases ที่ยกมากรณีพิเศษ

ปัญหาที่พบในกิจกรรมของคณาจารย์ เช่น ปัญหาในการจัดการเรียนการสอน

3 special cases

- การใส่ข้อมูลเข้าออกของ LinkList ไม่ซับซ้อนอีกต่อไป ไม่ซับซ้อนเหมือนตอน
- การลบสกรีนก็ค่อนข้างออกจากจากข้อมูล List ในภาพเพิ่มเติมอีกนิดๆ ของ List
- การลบข้อมูลในการรันตัว ๆ ไป เพราะไม่รู้จักชื่อเขาเลยก่อนหน้าคืออะไร

special cases

1. ทำ header node ที่ index 0
2. header เก็บ index 2 ขึ้นไป
3. ใช้ฟังก์ชัน `FindPrevious` เพื่อหา node ที่ index 1 ให้มันกับ header node

```
template <class Object>
class List;      // Incomplete declaration.
```

```
template <class Object>
class ListItr;      // Incomplete declaration.
```

ตำแหน่งของโหนด

```
template <class Object>
class ListNode {
```

```
ListNode( const Object & theElement = Object(),
        ListNode* n=NULL) : element(theElement), next(n) {}
```

```
Object element;
```

```
ListNode *next;
```

```
friend class List<Object>;
```

```
friend class ListItr<Object>;
```

} ;

List<int> มี 2 ขั้นตอน

ขั้นตอนที่ 1 ระบบปฏิบัติการคอมไพเลอร์ จะทำการสร้างตัวแปรออบเจกต์ a ขึ้นในหน่วยความจำ
ตามโครงสร้างคลาส List ซึ่งเอาค่า int จะถูกส่งผ่านไปให้ Class template
ชื่อ object ตรงไหนเป็นค่า object ในโครงสร้างคลาส List ก็จะถูกเปลี่ยนเป็นค่า
int สำหรับตัวแปรออบเจกต์ a

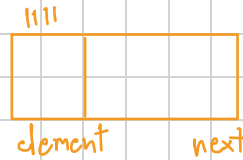
a

public

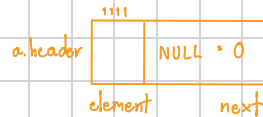
...

private

header =



ขั้นตอนที่ 2 ระบบปฏิบัติการจะไปทำงาน Constructor function ที่กำหนด List
... template < >



คำสั่ง new จะมีการทำงาน 2 อย่าง

1. บอกระบบปฏิบัติการคอมไพเลอร์ ให้ทำการจองพื้นที่ในหน่วยความจำตามโครงสร้าง Class ?
2. ไปทำงาน Constructure Function ในที่นี้ ก็คือฟังก์ชัน ?

ผลของคำสั่ง new ก็คือ address ที่จองได้ *** ที่นำไปเก็บพอยน์เตอร์ของเครื่อง =
เลข → ณ

int main

while ทำอะไรรอบ
จนไปจนกว่าจะเป็นเท็จ
* พอยเตอร์

List<int> a;

ListTr<int> b;

ตัวแปรออบเจกต์ a, b ไม่รับค่าตัวแปรเลขๆ

int x;

ตัวแปร x

x = 5; ✓

a = 5; ✗

a.remove(43);

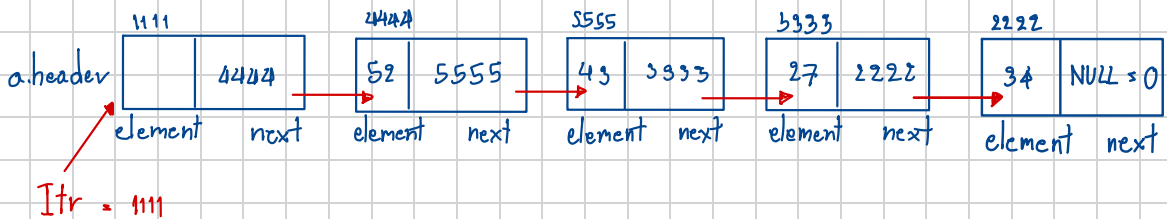
a.remove(27);

b = zeroth();

b.advance();

cout << a.header -> element; ✗ cout << b.retrieve(); ✓

ถึง header ออกมา cout ไม่ได้ เพราะเป็น private



Type declaration for link list node

```
template <class Object>
class List; // Incomplete declaration.

template <class Object>
class ListIter; // Incomplete declaration.

template <class Object>
class ListNode {
    Object element;
    ListNode *next;
};

friend class List<Object>;
friend class ListIter<Object>;
```

Handwritten notes in Thai: "ผ่านของโหนด" (node passed), "คือค่าที่อยู่ภายในโหนด" (value inside the node), "ถ้าไม่เพิ่มลิสต์หรือโหนด จะเป็นโหนดทั้งหมด" (if not adding list or node, it will be all nodes), "ค่าของโหนดจะเพิ่มเข้าไปในลิสต์" (node value will be added to the list).

Iterator class for linked lists

```
template <class Object>
class ListIter {
public:
    ListIter() : current( NULL ) {}
    bool isPastEnd() const { return current == NULL; }

    void advance() {
        if( !isPastEnd() ) current = current->next;
    }

    const Object & retrieve() const {
        if( isPastEnd() )
            throw BadIterator();
        return current->element;
    }

private:
    ListNode<Object> *current; // Current position
    ListIter(ListNode<Object> *theNode):current( theNode ) {}
    friend class List<Object>; // Grant access to constructor
};
```

Handwritten notes in Thai: "ปัจจุบัน" (current), "ตำแหน่งปัจจุบัน" (current position), "ถ้าไม่เพิ่มลิสต์หรือโหนด จะเป็นโหนดทั้งหมด" (if not adding list or node, it will be all nodes).

Iterator class for linked lists

```
template <class Object>
class ListIter {
public:
    ListIter() : current( NULL ) {}
    bool isPastEnd() const { return current == NULL; }

    void advance() {
        if( !isPastEnd() ) current = current->next;
    }

    const Object & retrieve() const {
        if( isPastEnd() )
            throw BadIterator();
        return current->element;
    }

private:
    ListNode<Object> *current; // Current position
    ListIter(ListNode<Object> *theNode):current( theNode ) {}
    friend class List<Object>; // Grant access to constructor
};
```

Handwritten notes in Thai: "ตำแหน่งปัจจุบัน" (current position), "ถ้าไม่เพิ่มลิสต์หรือโหนด จะเป็นโหนดทั้งหมด" (if not adding list or node, it will be all nodes).

List class interface

```
template <class Object>
class List {
public:
    List();
    List( const List & rhs );
    ~List();

    bool isEmpty() const;
    void makeEmpty();
    ListIter<Object> zeroth() const;
    ListIter<Object> first() const;
    void insert( const Object & x, const ListIter<Object> & p );
    ListIter<Object> find( const Object & x ) const;
    ListIter<Object> findPrevious( const Object & x ) const;
    void remove( const Object & x );
    const List & operator=( const List & rhs );

private:
    ListNode<Object> *header;
```

Some list one-liners

```

/* Construct the list */
template <class Object>
List<Object>::List()
{
    header = new ListNode<Object>;
}

/* Test if the list is logically empty */
template <class Object>
bool List<Object>::isEmpty() const
{
    return header->next == NULL;
}

```

5

find routine

```

/* Return iterator corresponding to the first
   node containing an item x. Iterator isPastEnd
   if item is not found. */

template <class Object>
ListItr<Object> List<Object>::find( const
Object & x ) const
{
    ListNode<Object> *itr = header->next;
    while( itr != NULL && itr->element != x )
        itr = itr->next;
    return ListItr<Object>( itr );
}

```

7

21

Some list one liners

```

/* Return an iterator representing the header node
   template <class Object>
   ListItr<Object> List<Object>::zeroth() const
   {
       return ListItr<Object>( header );
   }

/* Return an iterator representing the first node
   in the list. This operation is valid for empty
   lists. */
template <class Object>
ListItr<Object> List<Object>::first() const
{
    return ListItr<Object>( header->next );
}

```

6

Deletion routine for linked lists

```

/* Remove the first occurrence of an item x. */
template <class Object>
void List<Object>::remove( const Object & x )
{
    ListItr<Object> p = findPrevious( x );
    if( p.current->next != NULL )
    {
        ListNode<Object> *oldNode = p.current->next;
        p.current->next = p.current->next->next;
        delete oldNode;
    }
}

```

8

21

Deletion routine for linked lists

```

/* Remove the first occurrence of an item x. */
template <class Object>
void List<Object>::remove( const Object & x )
{
    ListItr<Object> p = findPrevious( x );
    if( p.current->next != NULL )
    {
        ListNode<Object> *oldNode = p.current->next;
        p.current->next = p.current->next->next;
        delete oldNode;
    }
}

```

9

21

findPrevious-the find routine for use with remove

```

/*Return iterator prior to the first node containing an
item x.

template <class Object>
ListItr<Object> List<Object>::findPrevious( const Object &
x ) const
{
    ListNode<Object> *itr = header;   itr = 1111
    while( itr->next != NULL && itr->next->element != x )   itr = 2
        itr = itr->next;   itr = 4444
    return ListItr<Object>( itr );
}

```

-> and's

25

findPrevious-the find routine for use with remove

```

/*Return iterator prior to the first node containing an
item x.

template <class Object>
ListItr<Object> List<Object>::findPrevious( const Object &
x ) const
{
    ListNode<Object> *itr = header;   itr = 4444
    while( itr->next != NULL && itr->next->element != x )   itr = 2
        itr = itr->next;   itr = 4444
    return ListItr<Object>( itr );
}

```

-> and's

26

findPrevious-the find routine for use with remove

```

/*Return iterator prior to the first node containing an
item x.

template <class Object>
ListItr<Object> List<Object>::findPrevious( const Object &
x ) const
{
    ListNode<Object> *itr = header;   itr = 1111
    while( itr->next != NULL && itr->next->element != x )   itr = 2
        itr = itr->next;   itr = 4444
    return ListItr<Object>( itr );
}

```

-> and's

25

findPrevious-the find routine for use with remove

```

/*Return iterator prior to the first node containing an
item x.

template <class Object>
ListItr<Object> List<Object>::findPrevious( const Object &
x ) const
{
    ListNode<Object> *itr = header;   itr = 4444
    while( itr->next != NULL && itr->next->element != x )   itr = 2
        itr = itr->next;   itr = 4444
    return ListItr<Object>( itr );
}

```

-> and's

26

```

/*Return iterator prior to the first node containing an
item x.

template <class Object>
ListItr<Object> List<Object>::findPrevious( const Object &
x ) const
{
    ListNode<Object> *itr = header;   itr = 4444
    while( itr->next != NULL && itr->next->element != x )   itr = 2
        itr = itr->next;   itr = 4444
    return ListItr<Object>( itr );
}

```

-> and's

/* Insert item x after p. */

```

template <class Object>
void List<Object>::insert( const Object & x,
const ListItr<Object> & p )
{
    if( p.current != NULL )
        p.current->next = new ListNode<Object>
            ( x, p.current->next );
}

```

Stack (เป็นข้อมูลแบบทวินาม) เข้าที่หลังออกก่อน

- 1 is empty

push มี 2 ขั้นตอน

1. เช็คค่า topOfStack ไปถึง 1

2. ค่า x ไปเก็บใน Stack[topOfStack]

ตรวจสอบข้อห้ามก่อนใส่

top return Stack[topOfStack]

ส่วนค่าที่เก็บที่ TopOfStack จะลบออก

ลบที่เก็บที่ Stack

Pop ค่าที่ลบออก topOfStack ลงไป 1

จงเขียนโปรแกรม (Statement)

เพื่อใส่ข้อมูล ไปเก็บที่ topOfStack --; an 2 ประโยค

-- topOfStack; และ topOfStack = topOfStack - 1;

top and pop 2 ขั้นตอน

1. ใส่ค่า

2. ลบค่าที่เก็บ / ถอดค่า

```
template <class Object>
```

```
class Stack {
```

```
public:
```

```
explicit Stack( int capacity = 10 );
```

เขียนตัวเก็บไว้ก่อนรับการใส่ข้อมูล / ใส่ข้อมูลแล้ว } 1. ใส่ข้อมูลก่อนการใส่ข้อมูล 2. ใส่ข้อมูลก่อนการใส่ข้อมูล

```
bool isEmpty( ) const;
```

```
bool isFull( ) const;
```

ไม่ได้นำมาใช้ data member int

```
const Object & top( ) const;
```

```
void makeEmpty( );
```

```
void pop( );
```

```
void push( const Object & x );
```

```
Object topAndPop( );
```

int ใส่ข้อมูลใน data member

```
private:
```

```
vector<Object> theArray;
```

```
int topOfStack;
```

```
};
```

เก็บข้อมูล

ถ้าใส่ข้อมูลก่อนการใส่ข้อมูล = 10

ถ้า void ไม่ต้องการเก็บค่า
ถ้า void มีการเก็บค่า

9

```
int main( )
```

Stack<int> a(7); มีโปรแกรมการใส่ข้อมูล 7 ครั้ง 3 ครั้ง 1, 2, 3

```
a.push(8);
```

```
a.push(4);
```

```
a.push(1);
```

```
a.push(9);
```

```
a.push(3);
```

```
a.push(5);
```

```
a.push(0);
```

```
a.push(0); ← u cant
```

```
a.pop();
```

```
a.pop();
```

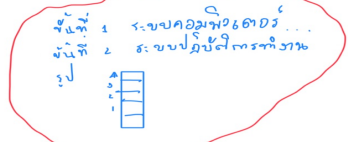
```
int x = 0;
```

```
x = a.top();
```

```
x = 3; return ค่าที่เก็บที่ topOfStack ที่ 3
```

```
x = a.topAndPop();
```

```
x = 3;
```



ดู เปรียบเทียบ การทำงานของระบบคอมพิวเตอร์ a ตามโครงสร้างของ Stack
ที่เก็บข้อมูลไว้ที่ topOfStack

```
return 0;
```

Stack <int> a(7), มี 3 ขั้นตอน

ขั้นที่ 1 ระบบปฏิบัติการคอมพิวเตอร์จะทำการสร้างตัวแปรของเจ็ท a ซึ่งในขณะเดียวกัน ถ้าใครสร้างคลาส Stack ชนิดของ int จะถูกส่งเข้าไปที่ Class template หรือ object ตอนไหนเป็นค่า object ในโครงสร้าง คลาส Stack จะถูกเปลี่ยนเป็นค่า int สำหรับตัวแปรของเจ็ท a

a

public

...

private

theArray =

topofStack =

ขั้นที่ 2 ระบบปฏิบัติการจะไปทำ Constructure function สำหรับ Stack
Stack <object> :: Stack (int capacity) : theArray (capacity)
{ topofStack = -1 ; }

ขั้นที่ 3

6	
5	
4	
3	
2	
1	
[0]	

theArray มี

topofStack = -1

void overflow ()

```
{  
    Cout << "Cannot push element into stack because stack is full";  
    exit (1);  
}
```

-- , ++ วิธีสังเกต

1. ถ้าใส่ตัวเลขหรือตัวแปรอื่นๆ ให้ใส่ค่าก่อน
2. ถ้าใส่ตัวเลขหรือตัวแปรอื่นๆ
- ถ้า -- ชุมบ่งชี้ให้เอาค่าไปก่อนก่อนลบค่า

```

template <class Object>
void Stack<Object>::push( const Object & x ) {
    if( isFull( ) ) throw Overflow( );
    theArray[ ++topOfStack ] = x;
}

/* Return top of stack
template <class Object>
const Object & Stack<Object>::top( ) const {
    if( isEmpty( ) ) throw Underflow( );
    return theArray[ topOfStack ];
}

/* Pop from stack
template <class Object>
void Stack<Object>::pop( ) {
    if( isEmpty( ) ) throw Underflow( );
    topOfStack--;
}

```

Handwritten notes:

- เพิ่มสิ่งใหม่* (Add new thing)
- ถ้า 2 ตัว หรือ 3 ตัว ก็ใช้ได้* (If 2 or 3 items, it's also fine)
- เริ่มค่าที่ 1 แล้วเพิ่มเรื่อย ๆ* (Start at 1 and increase)
- ค่าของ array - 1* (Array value - 1)
- บวกค่าเข้า* (Add value)
- topOfStack = 0*
- array ของ topOfStack & array ที่เก็บค่าไว้* (Array of topOfStack & array that stores values)
- topOfStack = 5*
- ถ้า 1 หรือ 2 ก็ใช้ได้* (If 1 or 2 items, it's also fine)
- ลบสิ่งเก่าออก* (Remove old thing)

6	6	0
5	5	5
4	4	3
3	3	9
2	2	1
1	1	4
0	0	8

6	0
5	5
4	3
3	9
2	1
1	4
0	8

```

/* Pop from stack and return value
template <class Object>
Object Stack<Object>::topAndPop( ) {
    if( isEmpty( ) )
        throw Underflow( );
    return theArray[ topOfStack-- ];
}

```

Handwritten notes:

- pop ค่าแล้วลบค่าตัว* (Pop value then subtract value)
- ลบสิ่งเก่าออกด้วยตัว theArray* (Remove old thing with theArray)
- ถ้า -- มันลบสิ่งเก่าออกก่อนคือลบค่า* (If -- it removes old thing first then subtract value)

Queue เหวคอย

- ใส่ข้อมูลปลายด้านหนึ่ง
- ลบข้อมูลปลายด้านหนึ่ง
- ทั่วหน้ารอบก่อน

Operations

Enqueue การใส่ข้อมูลเข้าที่อีกแนวหน้า *isEmpty*
 Dequeue การลบข้อมูลออกจากคิว ลบที่หน้าแถว

Array

เก็บข้อมูลตามหัวสุดท้ายสุด | เก็บขนาดคิว

Enqueue

1. เก็บค่าที่จนจนข้อมูลในคิวจนเต็มไป
2. เก็บค่าที่ตรงที่อีกแนวหน้า back ถึงไป 1
3. ลบค่า x ไปที่แนวหน้า back

vector แทน array

```

/* Construct the stack.
template <class Object>
Stack<Object>::Stack( int capacity ) : theArray( capacity ) {
    topOfStack = -1; }

```

```

/* Test if the stack is logically empty.

```

```

template <class Object>
bool Stack<Object>::isEmpty( ) const {
    return topOfStack == -1; }

```

```

/* Test if the stack is logically full.

```

```

template <class Object>
bool Stack<Object>::isFull( ) const {
    return topOfStack == theArray.size( ) - 1; }

```

```

/* Make the stack logically empty.

```

```

template <class Object>
void Stack<Object>::makeEmpty( ) {
    topOfStack = -1; }

```

```

/* Pop from stack and return value

```

```

template <class Object>
Object Stack<Object>::topAndPop( ) {
    if( isEmpty( ) )
        throw Underflow( );
    return theArray[ topOfStack-- ];
}

```

```

/* Push onto a stack

```

```

template <class Object>
void Stack<Object>::push( const Object & x ) {
    if( isFull( ) ) throw Overflow( );
    theArray[ ++topOfStack ] = x;
}

```

6	0
5	5
4	3
3	9
2	1
1	4
0	8

```

/* Return top of stack

```

```

template <class Object>
const Object & Stack<Object>::top( ) const {
    if( isEmpty( ) ) throw Underflow( );
    return theArray[ topOfStack ];
}

```

6	0
5	5
4	3
3	9
2	1
1	4
0	8

```

/* Pop from stack

```

```

template <class Object>
void Stack<Object>::pop( ) {
    if( isEmpty( ) ) throw Underflow( );
    topOfStack--;
}

```

```
int main()
```

```
{ Queue<int> a(7); // 2 ขั้นตอน ทำความด้วย / วาดรูป →
```

```
a.enqueue(5);
```

```
a.enqueue(8);
```

```
a.enqueue(4);
```

```
a.enqueue(3);
```

```
a.enqueue(2);
```

```
a.enqueue(1);
```

```
a.enqueue(6);
```

```
a.enqueue(9);
```

```
a.enqueue(2);
```

if is full แล้ว throw Overflow()

นำออกมา

```
int x;
```

```
x = a.dequeue();
```

```
x = 5; main front item
```

front

back → front →

7
6
4
3
2
1
0

```
return 0;
```

Queue<int> a(7); // 2 ขั้นตอน ทำความด้วย / วาดรูป

ขั้นที่ 1 รับปฎิบัติการคอนสตรัคเตอร์ จะทำการสร้างตัวแปรของเจกต์ a ขึ้น

ในขณะนั้นจะจองความจำที่คอนสตรัคเตอร์ Queue

ทีนี้ของ int จะถูกส่งเข้าไปที่ Class template & object

ตรงไหนเป็นค่า object ในคอนสตรัคเตอร์ Queue ก็จะเป็น

ถูกเปลี่ยนเป็นค่า int สำหรับตัวแปร ของเจกต์ a

a

public

...

private

theArray = ? address ของ Array ที่จองไว้

currentSize =

front =

back =

ขั้นที่ 2 รับปฎิบัติการจะไปรวม Constructure function ที่อยู่ใน Queue

template < class Object >

Queue< Object >:: Queue (int capacity):

theArray (capacity)

{ makeEmpty (); }

วาด

6	
5	
4	
3	
2	
1	
0	

1111

หลักการ wrap around ใน Queue

เมื่อ front หรือ back ไปถึงตำแหน่งท้ายของ array พอ queue ยังไม่เต็ม ให้ back หรือ front กลับมาที่ตำแหน่งที่ 0 ใหม่

```
/* Enqueue - Insert x into the queue.
template <class Object>
void Queue<Object>::enqueue( const Object & x ) { ①
    if( isFull( ) ) {
        throw Overflow( );
        increment( back ) = ++0
        theArray[ back ] = x; = 5
        ++currentSize++;
    }
```

```
/* Internal method to increment x with wraparound.
template <class Object>
void Queue<Object>::increment( int & x ) { ② ②
    if( ++x == theArray.size( ) ) {
        x = 0;
    }
```

```
/* Dequeue - Remove last recently inserted item
template <class Object>
Object Queue<Object>::dequeue( )
{
    if( isEmpty( ) )
        throw Underflow( );

    currentSize--;
    Object frontItem = theArray[ front ];
    increment( front );
    return frontItem;
}
```

Call by reference

&

ต้องแปร back จ.อยู่ในหน่วยความจำ ที่เดียวกับ x
back เปลี่ยนตาม x

Call by value

int x

ตัวแปร Back จ.อยู่ในหน่วยความจำ
คนละที่ กับ x
back ไม่เปลี่ยนค่าตาม x

```

template <class Object>
class Queue
{
public:
    explicit Queue( int capacity = 10 );

    bool isEmpty( ) const;
    bool isFull( ) const;
    const Object & getFront( ) const;
    void makeEmpty( );
    Object dequeue( );
    void enqueue( const Object & x );

private:
    the array vector<Object> theArray;
    int currentSize;
    int front;
    int back;
    data number do function void increment( int & x );
};

```

```

/*Construct the queue.
template <class Object>
Queue<Object>::Queue(int capacity):theArray(capacity)
{
    makeEmpty( );
}

/* Make the queue logically empty.
template <class Object>
void Queue<Object>::makeEmpty( )
{
    currentSize = 0;
    front = 0;
    back = -1;
}

```

```

/* Enqueue - Insert x into the queue.
template <class Object>
void Queue<Object>::enqueue( const Object & x ) { ①
    if( isFull( ) ) if full stack was 10 throw Overflow( ); void Overflow() {
    cout << "Cannot enqueue into Queue because Queue is full" << endl;
    increment( back ); back = 0
    theArray[ back ] = x; exit (1);
    ++currentSize++;
}

/* Internal method to increment x with wraparound.
template <class Object>
void Queue<Object>::increment( int & x ) { ② ③
    if( ++x == theArray.size( ) ) wrap around
        x = 0;
}

```

/* Dequeue - Remove last recently inserted item

```

template <class Object>
Object Queue<Object>::dequeue( )
{
    if( isEmpty( ) )
        throw Underflow( );

    currentSize--;
    Object frontItem = theArray[ front ]; ④
    increment( front );
    return frontItem;
}

```

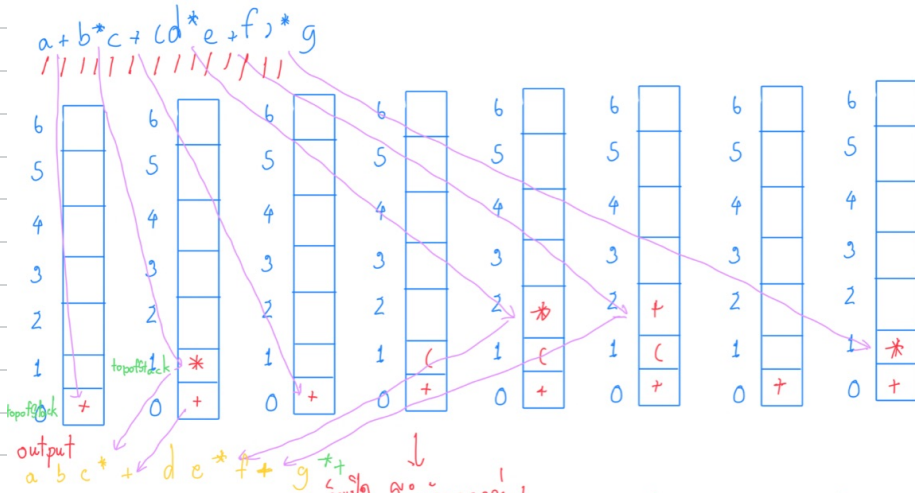
call by value
back
call by ref
back

Expression ที่ต้องแปลงให้เข้า postfix ก่อน

ลำดับความสำคัญของการคำนวณ

(), *, +

Infix to Postfix



- ระวังอย่าลืม c ที่ติดกับ a จาก a ที่ $+$
- ระวังอย่าลืม $+$ ที่ติดกับ a จาก a ที่ $+$
- push $+$ ที่ไปบน $*$ ไปได้ $+$ $+$ pop
- เมื่อเจอ $)$ ระวังอย่าลืม push $+$ ที่ไปบน $*$ แต่จะ pop stack ก่อน จนกว่าจะเจอ $+$ แล้ว push $+$ และจะ push $+$ ไปบน $*$
- ระวังอย่าลืม $+$ ที่ติดกับ a จาก a ที่ $+$

แทนค่า

$$a + b * c + (d * e + f) * g$$

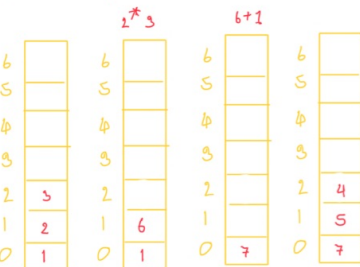
'infix' $= 1 + 2 * 3 + 26 * 7 = 1 + 6 + 26 * 7 = 1 + 6 + 182 = 189$

$$1 + 2 * 3 + (4 * 5 + 6) * 7$$

$$a \ b \ c \ * \ + \ d \ e \ * \ f \ + \ g \ * \ +$$

'postfix' $= 189$

$$1 \ 2 \ 3 \ * \ + \ 4 \ 5 \ * \ 6 \ + \ 7 \ * \ +$$



Expression Tree

$$(a + (b * c)) + ((d * e) + f) * g$$

//////////

6		6		6		6		6		6		6		6		6	
5		5		5		5		5		5		5		5		5	
4		4		4		4		4		4		4		4		4	
3		3		3		3	*	3		3		3		3		3	(
2		2		2	(	2	(	2		2		2		2	(	2	(
1		1	+	1	+	1	+	1	+	1		1	(	1	(	1	(
0	(	0	(	0	(	0	(	0	(	0	+	0	+	0	+	0	+

6		6		6		6		6		6	
5		5		5		5		5		5	
4	*	4		4		4		4		4	
3	(	3		3	+	3		3		3	
2	(	2	(	2	(	2		2	*	2	
1	(	1	(	1	(	1	(	1	(	1	
0	+	0	+	0	+	0	+	0	+	0	+

Output

$$a b c * + d e * f + g * +$$

infix $(a + (b * c)) + ((d * e) + f) * g$

$$(1 + (2 * 3)) + ((4 * 5) + 6) * 7 = (1 + 6) + ((20 + 6) * 7) = 7 + (26 * 7) = 189$$

prefix $a b c * + d e * f + g * +$

~~1~~ ~~2~~ ~~3~~ * ~~4~~ ~~5~~ * ~~6~~ + ~~7~~ * ~~+~~

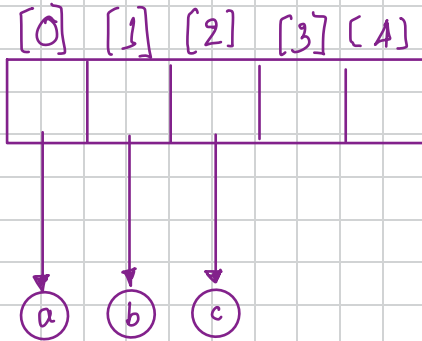
6		6		6		6		6		6		6	
5		5		5		5		5		5		5	
4		4		4		4		4		4		4	
3		3		3		3		3		3		3	
2	3	2		2		2	5	2		2	6	2	
1	2	1	6	1		1	4	1	20	1	20	1	26
0	1	0	1	0	7	0	7	0	7	0	7	0	7

6		6		6	
5		5		5	
4		4		4	
3		3		3	
2	7	2		2	
1	26	1	182	1	
0	7	0	7	0	189

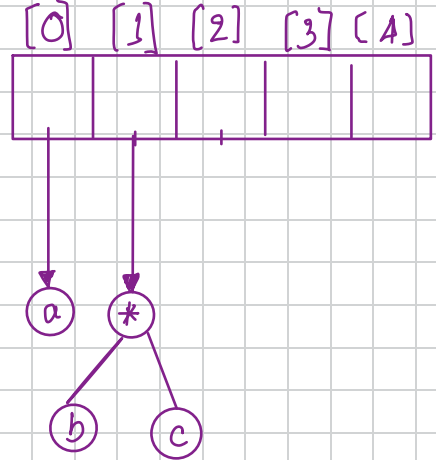
in Expression Tree construction $(a + (b * c)) + (((d * e) + f) * g)$

prefix is $a b c * + d e * f + g * +$
 // // // // //

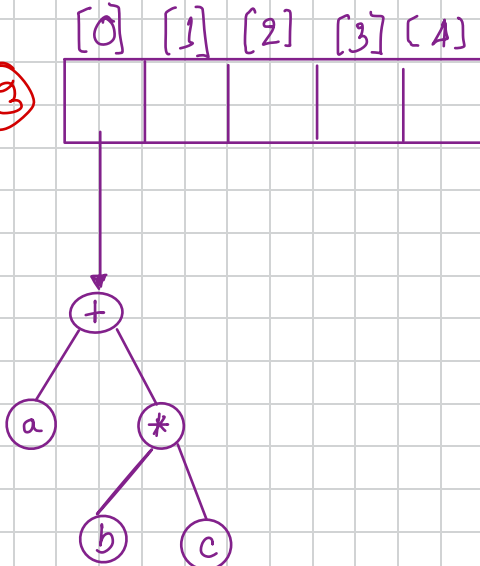
①

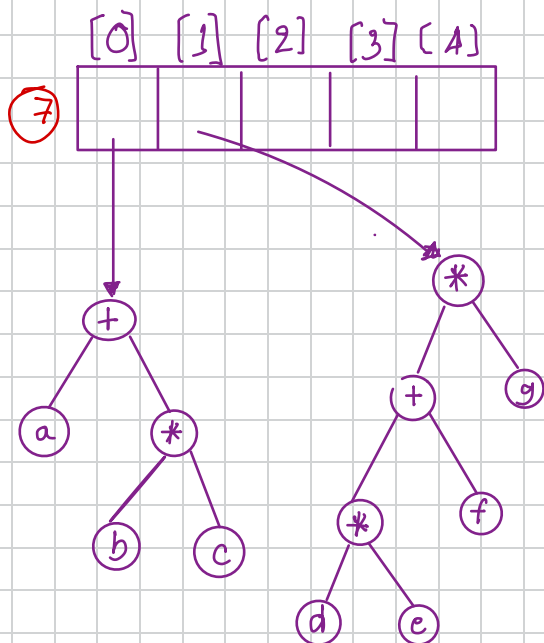
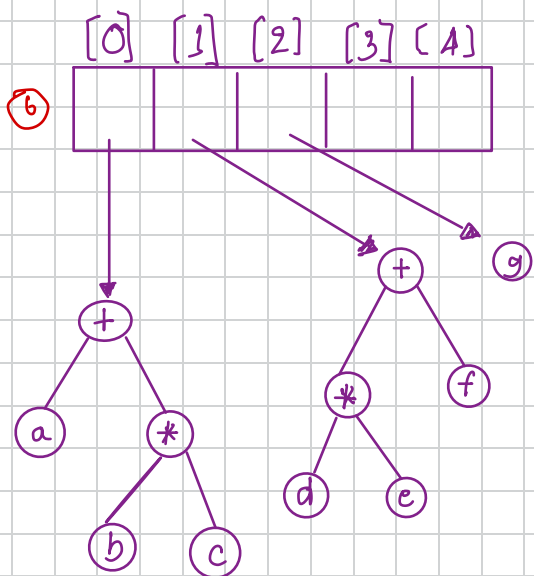
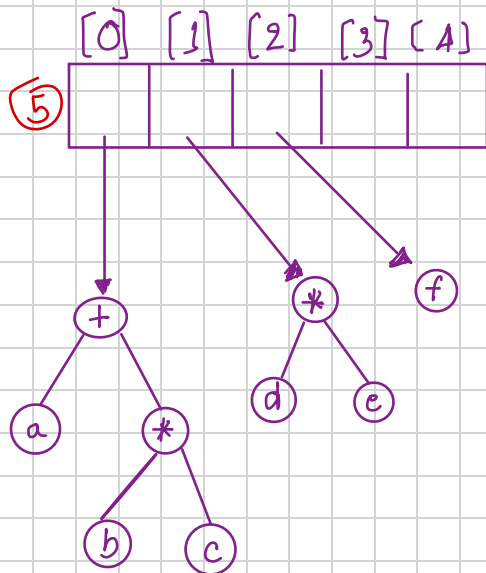
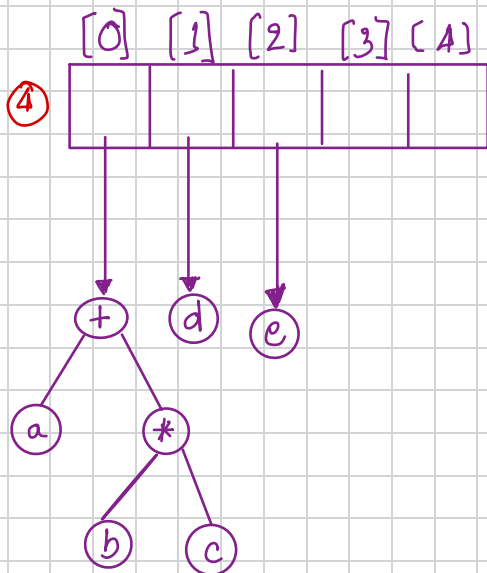


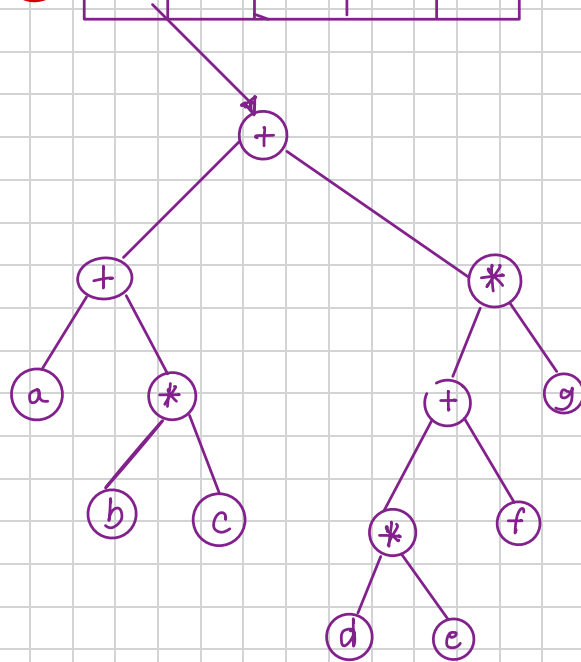
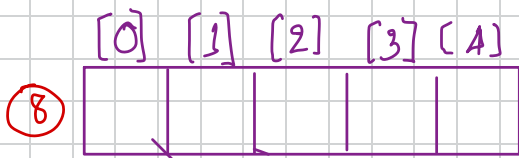
②



③





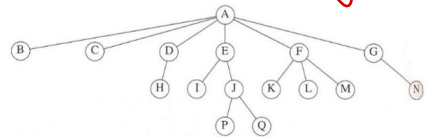


Trees รวมกันหมด

edges เส้น / root node ไม่มี parent
path บัดนี้
การคลิก ย้อนไปบางจุด
การวนซ้ำ บางครั้งต้องรู้

จงเขียนเส้นทาง (path) จากโหนด A ไป Q
A, E, J, Q

จงเขียนเส้นทาง (path) จากโหนด Q ไป A
ไม่มีเส้นทางจาก Q ไป A

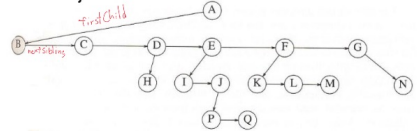


6

Implementation of Trees

3. Each node contains a linked list of the children

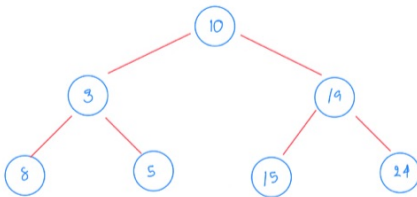
```
struct TreeNode {
    Object element;
    TreeNode *firstChild;
    TreeNode *nextSibling;
};
```



7

Tree Traversals ครอบคลุมทั้ง tree

Pre Order ข้างหน้า วนซ้ำ ซ้าย ขวา
In Order ข้างใน วนซ้ำ ซ้าย ขวา
Post Order ข้างหลัง วนซ้ำ ซ้าย ขวา



Pre Order : 10, 3, 8, 5, 19, 15, 24

In Order : 8, 3, 5, 10, 15, 19, 24

Post Order : 8, 5, 3, 15, 24, 19, 10

Binary Search Trees

ใช้ในการค้นหาข้อมูล

ข้อมูลจะถูกจัดเก็บและค้นหาตามลำดับ

1. node มีลูกได้ไม่เกิน 2

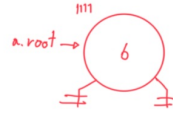
ซ้ายต้องน้อยกว่าโหนด / ขวาต้องมากกว่าโหนด

```
int main()
```

```
{ BinarySearchTree <int> a(999);
```

```
  a.insert(6); // สร้างโหนด insert
  a.insert(5);
```

2. ง่าย 100% น. 9.1 คือด้วย



2. ง่าย 100%

ขั้นตอนที่ 1: เราขอยกตัวอย่างการคอมพิวเตอรื จะทำการสร้างตัวประกอบเจ็ดตัว a ซึ่งในตัวอย่างจะมาจากคณิศรสร้างคลาส BinarySearchTree ซึ่งเอาข้อมูล int จะถูกส่งเข้าไปที่ class template ชื่อ Comparable ตรงไหนที่บอกว่า Comparable ในโครงสร้างคลาส BinarySearchTree ก็จะถูกเปลี่ยนเป็นค่า int ถ้ารับตัวแปร อยบเจ็ดตัว a

```
public
```

```
...
```

```
private
```

```
root = ?
```

```
ITEM_NOT_FOUND = ?
```

ขั้นตอนที่ 2: เราขอยกตัวอย่างการคอมพิวเตอรื Constructure function ที่รับ BinarySearchTree

BinarySearchTree the public อยบเจ็ดตัว BinarySearchTree

```
1. template < class Comparable >
```

```
2. BinarySearchTree < Comparable > ..
```

```
3. BinarySearchTree c const Comparable & notFound )
```

```
{
```

```
    root = NULL;
```

```
    ITEM_NOT_FOUND = notFound;
```

```
}
```

insert, remove

```
{
```

```
    insert(x, root); // หมายเหตุ 34
```

```
}
```

คณิศรเอาไปใส่ public หรือ private จะใส่ใน private ทั้งหมด

Binary Search Trees

```
template <class Comparable>
class BinarySearchTree
```

```
template <class Comparable>
```

```
class BinaryNode {
    Comparable element;
    BinaryNode *left;
    BinaryNode *right;
```

สามารถใส่ public หรือ private
จะเพิ่ม private หรืออะไร

```
BinaryNode( const Comparable & theElement,
            BinaryNode *lt, BinaryNode *rt ) :
```

```
    element( theElement ), left( lt ), right( rt ) { }
```

```
friend class BinarySearchTree <Comparable>
```

```
};
```

25

เขียนฟังก์ชัน insert

```
template <class Comparable >
void BinarySearchTree < Comparable >::
```

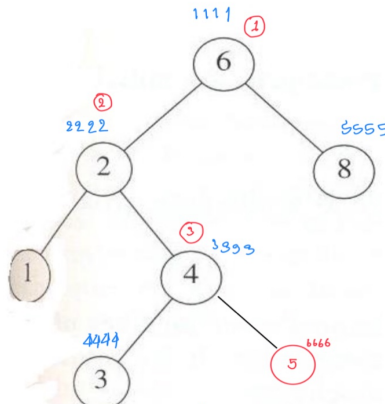
```
insert ( const Comparable & x)
{
    insert ( x, root );
```

เวลาเขียนฟังก์ชัน ให้ลบ void
ออกจากหน้า insert, remove

```
void insert( const Comparable & x );
```

find

เขียนฟังก์ชัน find



- ① สมมติว่า $x = 9, t = 1111$
insert (x, t → left);
5, 2222
- ② สมมติ 2 $x = 5, t = 2222$
insert (x, t → right);
5, 3333
- ③ สมมติ 3 $x = 5, t = 3333$
insert (x, t → right);
5, null
- ④ สมมติ 4 $x = 5, t = null$ ให้ใส่ค่า 25
5, null call by reference

การหาตำแหน่ง insert ให้ private แล้ว 3p

4.3.4 remove

As is common with many data structures, the hardest operation is deletion. Once we have found the node to be deleted, we need to consider several possibilities.

① If the node is a leaf, it can be deleted immediately. ② If the node has one child, the node can be deleted after its parent adjusts a link to bypass the node (we will draw the link directions explicitly for clarity). See Figure 4.24.

③ The complicated case deals with a node with two children. The general strategy is to replace the data of this node with the smallest data of the right subtree (which is easily found) and recursively delete that node (which is now empty). Because the smallest node in the right subtree cannot have a left child, the second remove is an easy one. Figure 4.25

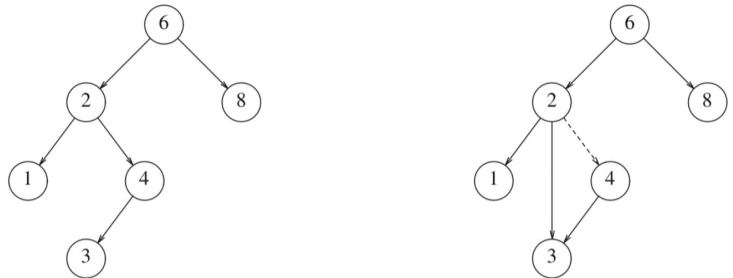
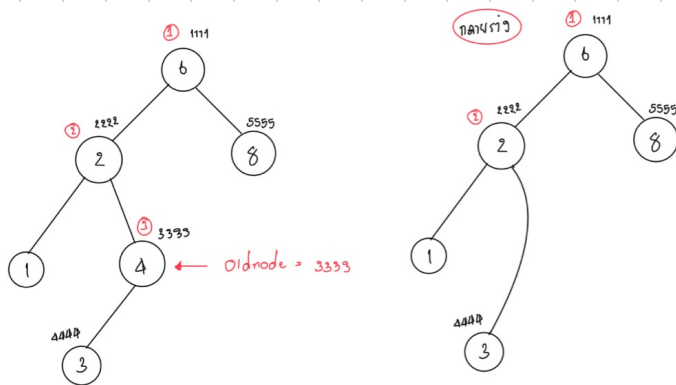


Figure 4.24 Deletion of a node (4) with one child, before and after



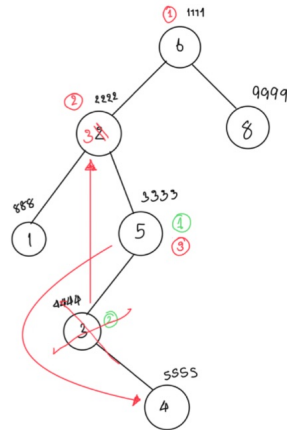
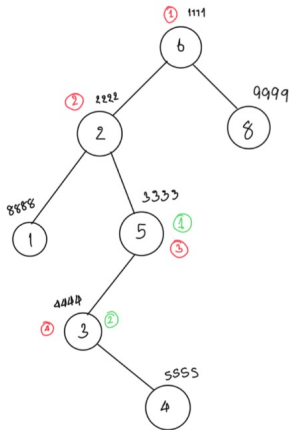
remove node with one child
 min/max remove private node

① $x = 4, t = 1111$
 remove (x, t → left);

② $x = 4, t = 2222$
 remove (x, t → right);

③ $x = 4, t = 3333$
 → call by reference

$t = (t \rightarrow \text{left} \neq \text{null}) ? t \rightarrow \text{left} : t \rightarrow \text{right};$
 3333 ? 1111 : 2222



remove node with two child so find its in order successor and replace it with it and then remove the private node

① solution $x = 2, t = 1111$
 $\text{remove}(x, t \rightarrow \text{left});$
 $t = 2222$

② solution $x = 2, t = 2222$
 $t \rightarrow \text{element} = \text{findMin}(t \rightarrow \text{right}) \rightarrow \text{element};$

return 3
 $\text{remove}(t \rightarrow \text{element}, t \rightarrow \text{right});$

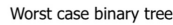
③ solution $x = 3, t = 3333$
 $\text{remove}(x, t \rightarrow \text{left});$
 $t = 4444$

④ solution $x = 3, t = 4444$
 $\text{deleteNode} = 4444$
 $\text{new oldnode} \rightarrow \text{call of referent}$

$t = (t \rightarrow \text{left} \neq \text{null}) ? t \rightarrow \text{left} : t \rightarrow \text{right};$
 $\text{return } t$

① solution $t = 5555$
 $\text{return findMin}(t \rightarrow \text{left});$
 $t = 4444$

② solution $t = 4444$
 $\text{return } t; t = 4444 \text{ don't need findMin}$



12

```
template <class Comparable>
class BinarySearchTree
{
public:
    explicit BinarySearchTree( const Comparable & notFound);
    BinarySearchTree( const BinarySearchTree & rhs );
    ~BinarySearchTree ( );

    const Comparable & findMin() const;
    const Comparable & findMax() const;
    const Comparable & find (const Comparable & x) const;

    bool isEmpty() const;
    void printTree() const;
    void makeEmpty();

    void insert( const Comparable & x );
    void remove( const Comparable & x );
    const BinarySearchTree & operator=
        ( const BinarySearchTree & rhs );

private:
    BinaryNode < Comparable > *root;
    const Comparable ITEM_NOT_FOUND;
```

```
private:
    const Comparable & elementAt
        ( BinaryNode<Comparable> *t ) const;

    void insert ( const Comparable & x,
                  BinaryNode<Comparable> * & t ) const;

    void remove ( const Comparable & x,
                  BinaryNode<Comparable> * & t ) const;

    BinaryNode<Comparable> * findMin
        ( BinaryNode<Comparable> *t ) const;
    BinaryNode<Comparable> * findMax
        ( BinaryNode<Comparable> *t ) const;
    BinaryNode<Comparable> * find ( const Comparable & x,
                                    BinaryNode<Comparable> *t ) const;

    void makeEmpty ( BinaryNode<Comparable> * & t ) const;
    void printTree ( BinaryNode<Comparable> *t ) const;

    BinaryNode<Comparable> * clone
        ( BinaryNode<Comparable> *t ) const;
};
```



find

```
/* Find item x in the tree - Return the matching item or
   ITEM_NOT_FOUND if not found. */

template <class Comparable>
const Comparable & BinarySearchTree<Comparable>::find( const
    Comparable & x ) const
{
    return elementAt( find( x, root ) );
}

template <class Comparable> const Comparable &
    BinarySearchTree<Comparable>::elementAt(
        BinaryNode<Comparable> *t ) const
{
    if( t == NULL ) return ITEM_NOT_FOUND;
    else return t->element;
}
```



find

```
/* Find item x in the tree - Return the node containinig the
   matching item or NULL if not found. */

template <class Comparable>
BinaryNode<Comparable> *
    BinarySearchTree<Comparable>::find
        ( const Comparable & x, BinaryNode<Comparable> *t ) const;
{
    if (t==NULL) return NULL;
    else if (x < t->element)
        return find( x, t->left );
    else if (t->element < x)
        return find( x, t->right );
    else
        return t;
}
```

Recursive implementation of findMin

```
/* Internal method to find the smallest item in a subtree t. */
```

```
template <class Comparable> BinaryNode<Comparable> *  
BinarySearchTree<Comparable>::findMin(  
    BinaryNode<Comparable> *t ) const  
{  
    if( t == NULL ) return NULL;  
    if( t->left == NULL ) return t;  
    return findMin( t->left );  
}
```

Non-recursive implementation of findMax

```
template <class Comparable> BinaryNode<Comparable> *  
BinarySearchTree<Comparable>::findMax(  
    BinaryNode<Comparable> *t ) const  
{  
    if( t != NULL )  
        while( t->right != NULL )  
            t = t->right;  
    return t;  
}
```

insert into a binary search tree

```
template <class Comparable>  
void BinarySearchTree<Comparable>::insert( const  
    Comparable & x, BinaryNode<Comparable> * &t ) const  
{  
    if( t == NULL )  
        t = new BinaryNode<Comparable>( x, NULL, NULL );  
    else if( x < t->element ) น้อยกว่า  
        insert( x, t->left );  
    else if( t->element < x )  
        insert( x, t->right );  
    else ; // Duplicate; do nothing  
}
```

Remove

```
template <class Comparable> void  
BinarySearchTree<Comparable>::remove( const Comparable &  
    x, BinaryNode<Comparable> * &t ) const  
{  
    if( t == NULL ) return;  
    if( x < t->element ) remove( x, t->left );  
    else if( t->element < x ) remove( x, t->right );  
    else if( t->left != NULL && t->right != NULL ) {  
        t->element = findMin( t->right )->element;  
        remove( t->element, t->right );  
    }  
    else {  
        BinaryNode<Comparable> *oldNode = t;  
        t = ( t->left != NULL ) ? t->left : t->right;  
        delete oldNode;  
    }  
}
```

นางสาวอัมพรรัตน์ ขอดำสาร รหัสนักศึกษ 506021620113

จัด 20113 สาขา input เพื่อบันทึกตัวได้ดังนี้ 6, 2, 0, 1, 1, 3

ทำการใส่ข้อมูลเข้าไปในช่องประกอบจุด a ตามโครงสร้างคลาส List ตามลำดับ
เพื่อให้ได้ผลลัพธ์จากข้อไปออก ดังนี้ 0, 1, 1, 2, 3, 6

```
int main ()
```

```
{ List<int> a;  
  List<int> b;
```

```
  b = a.zereth(); - เลขที่ ①  
                     ในหน่วย
```

```
  a.insert(3, b);
```

```
  a.insert(2, b);
```

```
  a.insert(1, b);
```

```
  a.insert(1, b);
```

```
  a.insert(0, b);
```

```
  b.advance(1);
```

```
  b.advance(1);
```

```
  b.advance(1);
```

```
  b.advance(1);
```

```
  b.advance(1);
```

```
  a.insert(6, b);
```

```
  printList(a);
```

```
  return 0;
```

```
}
```

0 1 1 2 3 6